

Importance Sampling for Option Pricing

Suman Mishra

25 September 2025

Introduction

Importance Sampling is a variance reduction technique for Monte Carlo simulations in option pricing. It improves efficiency by focusing sampling on important regions of the distribution.

Intuition

Consider a European call payoff:

$$\max(S_T - K, 0),$$

where S_T is the stock price at maturity and K the strike price.

For out-of-the-money options, the payoff is zero most of the time, causing high variance in standard Monte Carlo estimates.

Importance sampling addresses this by sampling from a biased distribution that makes rare, important (in-the-money) events more likely, correcting with weights to maintain unbiasedness.

Theoretical Framework

Expectation of Payoff

The option price can be written as

$$\mathbb{E}[\max(S_T - K, 0)] = \mathbb{E}[(S_T - K)\mathbf{1}_{\{S_T > K\}}].$$

Decomposing on an event K :

$$\mathbb{E}[(S_T - K)\mathbf{1}_{\{S_T > K\}}] = \mathbb{E}[(S_T - K)\mathbf{1}_{\{K > S_T\}}] + \mathbb{E}[(S_T - K)\mathbf{1}_{\{K \leq S_T\}}].$$

If $\{K > S_T\}$ is rare, then

$$\mathbb{E}[(S_T - K)\mathbf{1}_{\{K > S_T\}}] \approx 0,$$

meaning very few samples contribute to the payoff, increasing variance.

Importance sampling shifts the distribution so $\{K \leq S_T\}$ occurs more often, reducing variance by weighting important outcomes more.

Monte Carlo Estimators

We want to estimate

$$\mu = \mathbb{E}_f[g(X)] = \int g(x)f(x) dx,$$

where $X \sim f$, and g is the payoff function.

Standard MC: Sample $X_i \sim f$, estimate

$$\hat{\mu}_{MC} = \frac{1}{N} \sum_{i=1}^N g(X_i).$$

Importance Sampling: Choose h with $h(x) > 0$ where $g(x)f(x) \neq 0$, rewrite

$$\mu = \int g(x) \frac{f(x)}{h(x)} h(x) dx = \mathbb{E}_h \left[g(X) \frac{f(X)}{h(X)} \right].$$

Sample $X_i \sim h$, estimate

$$\hat{\mu}_{IS} = \frac{1}{N} \sum_{i=1}^N g(X_i) w(X_i), \quad w(x) = \frac{f(x)}{h(x)}.$$

The estimator is unbiased; the goal is to choose h minimizing variance.

We choose the importance sampling density h by shifting it with parameter μ because the variance of the estimator is minimized when $h(x)$ closely approximates the integrand $g(x)f(x)$.

Since the optimal importance distribution is proportional to $g(x)f(x)$, selecting h centered at μ , where $g(x)f(x)$ attains high values, increases the likelihood of sampling important outcomes that contribute most to the expectation. This reduces estimator variance by focusing sampling effort on regions with significant payoff contributions.

Application: European Call Option

Under the Black-Scholes model, the stock price at maturity T is given by

$$S_T = S_0 \exp \left(\left(r - \frac{\sigma^2}{2} \right) T + \sigma W_T \right),$$

where $W_T \sim \mathcal{N}(0, T)$ is a Brownian motion at time T .

Defining the standardized variable

$$Z = \frac{W_T}{\sqrt{T}} \sim \mathcal{N}(0, 1),$$

we can rewrite S_T as

$$S_T = S_0 \exp \left(\left(r - \frac{\sigma^2}{2} \right) T + \sigma \sqrt{T} Z \right).$$

Since the payoff is zero for many values of Z (out-of-the-money), importance sampling shifts the distribution of Z to $\mathcal{N}(\mu, 1)$ to increase sampling in the payoff region.

Optimal Shift μ

The optimal shift μ for importance sampling is chosen so that the transformed variable targets the critical threshold where the option payoff becomes positive. Specifically, solving the equation for the terminal stock price at the strike:

$$S_T = K \implies \mu = \frac{\ln \left(\frac{K}{S_0} \right) - \left(r - \frac{\sigma^2}{2} \right) T}{\sigma \sqrt{T}}.$$

This choice of μ is effective because the importance sampling distribution h closely approximates the product of the original density $f(x)$ and the payoff function $g(x)$, minimizing the estimator's variance. The shifted distribution h attains its maximum near μ , corresponding to the event:

$$S_T > K \implies Z > \frac{\ln \left(\frac{K}{S_0} \right) - \left(r - \frac{\sigma^2}{2} \right) T}{\sigma \sqrt{T}} = z^*,$$

where Z is the standard normal variable driving S_T .

Likelihood Ratio

For samples $Z_i \sim \mathcal{N}(\mu, 1)$, the weight is

$$w(Z_i) = \frac{\varphi(Z_i; 0, 1)}{\varphi(Z_i; \mu, 1)} = \exp\left(-\mu Z_i + \frac{\mu^2}{2}\right),$$

where $\varphi(\cdot; m, s^2)$ is the normal PDF.

Estimator

The stock price for each sample is

$$S_T^{(i)} = S_0 \exp\left(\left(r - \frac{\sigma^2}{2}\right)T + \sigma\sqrt{T}Z_i\right),$$

with payoff

$$g(Z_i) = \max(S_T^{(i)} - K, 0),$$

and importance sampling estimator

$$\hat{\mu}_{IS} = e^{-rT} \frac{1}{N} \sum_{i=1}^N g(Z_i) w(Z_i).$$

Exercise 1: Importance Sampling for European Options with Varying Strike

Listing 1: Importance Sampling Monte Carlo for European Options with Varying Strike

```
1 import numpy as np
2 import time
3
4 # Parameters
5 S0 = 60
6 T = 1.0
7 r = 0.03
8 sigma = 0.3
9 n = 1000000 # Number of samples
10 Ks = np.arange(60, 121, 10)
11
12 def monte_carlo_importance_sampling(S0, K, r, sigma, T, n):
13     mu_call = (np.log(K / S0) + 0.5 * T * (sigma**2 - 2 * r)) / (sigma * np.sqrt(T))
14     mu_put = (np.log(S0 / K) + 0.5 * T * (sigma**2 - 2 * r)) / (sigma * np.sqrt(T))
15
16     Z_call = np.random.normal(loc=mu_call, scale=1, size=n)
17     ST_call = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z_call)
18     payoff_call = np.maximum(ST_call - K, 0)
19     likelihood_call = np.exp(-mu_call * Z_call + 0.5 * mu_call**2)
20     discounted_payoff_call = np.exp(-r * T) * likelihood_call * payoff_call
21     price_call = np.mean(discounted_payoff_call)
22     se_call = np.std(discounted_payoff_call, ddof=1) / np.sqrt(n)
23     pct_se_call = (se_call / price_call) * 100 if price_call > 0 else 0
24
25     Z_put = np.random.normal(loc=mu_put, scale=1, size=n)
26     ST_put = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z_put)
```

```

27 payoff_put = np.maximum(K - ST_put, 0)
28 likelihood_put = np.exp(-mu_put * Z_put + 0.5 * mu_put**2)
29 discounted_payoff_put = np.exp(-r * T) * likelihood_put * payoff_put
30 price_put = np.mean(discounted_payoff_put)
31 se_put = np.std(discounted_payoff_put, ddof=1) / np.sqrt(n)
32 pct_se_put = (se_put / price_put) * 100 if price_put > 0 else 0
33
34 return price_call, pct_se_call, price_put, pct_se_put
35
36 results = []
37
38 for K in Ks:
39     price_call, pct_se_call, price_put, pct_se_put =
40         monte_carlo_importance_sampling(S0, K, r, sigma, T, n)
41     results.append((K, pct_se_call, pct_se_put))
42
43 print(f"{'K':>10}{'%SE Call':>12}{'%SE Put':>12}")
44 for K, se_call, se_put in results:
45     print(f"{K:10d}{'se_call:12.4f'}{'se_put:12.4f'}")

```

Error Comparison

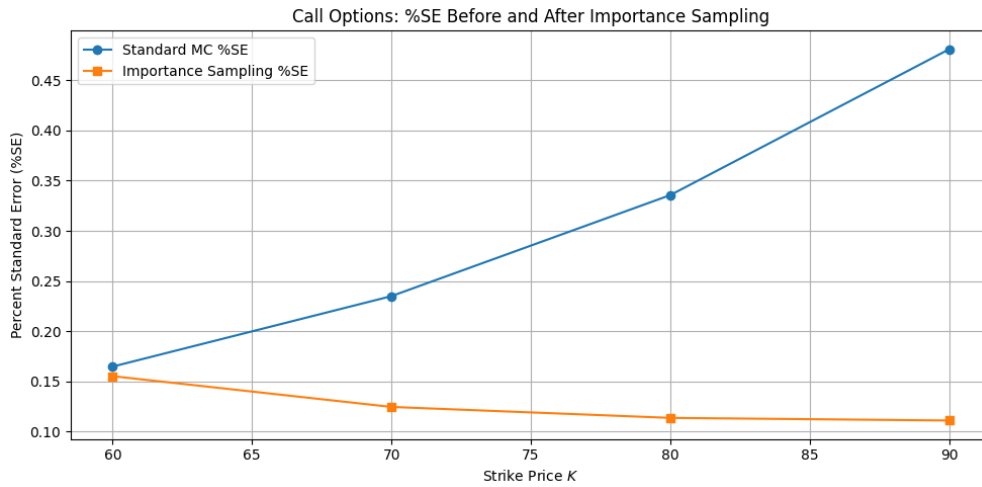


Figure 1: Call Options: %SE Before and After Importance Sampling

K	%SE Call	%SE Put
40	0.6193	3.6264
50	0.2438	0.4765
60	0.1550	0.1402
70	0.1248	0.0531
80	0.1138	0.0416
90	0.1111	0.0850
100	0.1122	0.1561
110	0.1147	0.2710
120	0.1182	0.4813

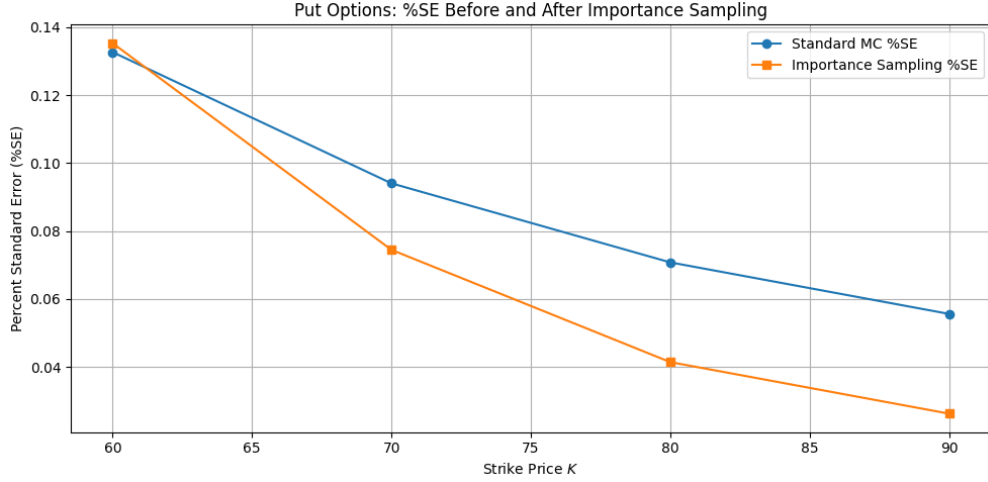


Figure 2: Put Options: %SE Before and After Importance Sampling

Exercise 2: Importance Sampling for Asian Call Option

The payoff depends on the average price

$$A_T = \frac{1}{M} \sum_{j=1}^M S_{t_j}.$$

Simulation

Divide T into M steps of $\Delta t = T/M$. At each step,

$$S_{t_j} = S_{t_{j-1}} \exp \left(\left(r - \frac{1}{2} \sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z_j \right),$$

with $Z_j \sim \mathcal{N}(\mu, 1)$, applying a constant shift μ .

Likelihood Ratio

Because of independence, the joint probability density function (pdf) of the whole vector Z is the product of the individual densities. The likelihood ratio $w(Z)$ is the ratio of the original joint pdf to the shifted joint pdf, i.e.:

$$w(Z) = \frac{\prod_{j=1}^M \varphi(Z_j; 0, 1)}{\prod_{j=1}^M \varphi(Z_j; \mu, 1)} = \prod_{j=1}^M \frac{\varphi(Z_j; 0, 1)}{\varphi(Z_j; \mu, 1)}.$$

Which simplifies to:

$$w(Z) = \frac{\varphi(Z_j; 0, 1)}{\varphi(Z_j; \mu, 1)} = \exp \left(-\mu Z_j + \frac{\mu^2}{2} \right).$$

Estimator

Payoff:

$$g(Z) = \max(0, A_T - K).$$

Estimator:

$$\hat{\mu}_{IS} = e^{-rT} \frac{1}{N} \sum_{i=1}^N g(Z_i) w(Z_i).$$

Listing 2: Monte Carlo Pricing for Asian Call Options with and without Importance Sampling

```

1 import numpy as np
2 import time
3 import matplotlib.pyplot as plt
4
5 def monte_carlo_asian_call(S0, K, r, sigma, T, n, m=365): #for 365 days
6     start_time = time.time()
7     dt = T / m
8
9     Z = np.random.normal(size=(n, m))
10    W = np.cumsum(Z * np.sqrt(dt), axis=1)
11
12    time_grid = np.linspace(dt, T, m)
13    exponent = (r - 0.5 * sigma**2) * time_grid + sigma * W
14    S_paths = S0 * np.exp(exponent)
15
16    S_avg = np.mean(S_paths, axis=1)
17    payoff = np.maximum(S_avg - K, 0)
18
19    discounted_payoff = np.exp(-r * T) * payoff
20
21    price_estimate = np.mean(discounted_payoff)
22    se = np.std(discounted_payoff, ddof=1) / np.sqrt(n)
23    pct_se = (se / price_estimate) * 100
24    runtime = time.time() - start_time
25
26    return price_estimate, pct_se, runtime
27
28 # Define parameters
29 S0 = 60
30 K = 65
31 r = 0.03
32 sigma = 0.3
33 T = 1.0
34 m = 50
35
36 ns = [10000, 20000, 50000, 100000]
37
38 results_standard = []
39 results_importance = []
40
41 for n in ns:
42     price_std, pct_se_std, runtime_std = monte_carlo_asian_call(S0, K, r, sigma, T
43         , n, m)
44     price_imp, pct_se_imp, runtime_imp =
45         monte_carlo_importance_sampling_asian_call(S0, K, r, sigma, T, n, m)
46     results_standard.append((n, price_std, pct_se_std, runtime_std))
47     results_importance.append((n, price_imp, pct_se_imp, runtime_imp))
48
49 # Print results
50 print(f"{'Sample_Size(n)':>12}{'Standard_MCPrice':>18}{'Standard_MC%SE':>16}{'
51     Importance_SamplingPrice':>27}{'Importance_Sampling%SE':>24}")
52 print("-" * 105)
53 for i in range(len(ns)):
54     print(f"{results_standard[i][0]:12}{results_standard[i][1]:18.4f}{
55         results_standard[i][2]:16.4f}{results_importance[i][1]:27.4f}{
56         results_importance[i][2]:24.4f}")

```

```

53 # Plot comparison
54 plt.figure(figsize=(10,6))
55 plt.plot(ns, [r[2] for r in results_standard], label='Standard MC %SE', marker='o'
56 )
57 plt.plot(ns, [r[2] for r in results_importance], label='Importance Sampling MC %SE', marker='x'
58 )
59 plt.xlabel('Sample Size (n)', fontsize=14)
60 plt.ylabel('Percentage Standard Error (%SE)', fontsize=14)
61 plt.title('Comparison of %SE: Standard MC vs Importance Sampling MC', fontsize=16)
62 plt.legend()
63 plt.grid(True)
64 plt.xscale('log')
65 plt.show()

```

Table 1: Asian Call Option Pricing: Standard MC vs Importance Sampling MC

Sample Size n	Standard MC Price	Standard MC %SE	Importance Sampling Price	Importance Sampling %SE
10000	2.6364	2.1551	2.6403	1.5597
20000	2.7062	1.4860	2.6411	1.1151
50000	2.5864	0.9638	2.6086	0.7089
100000	2.6572	0.6746	2.6430	0.4980

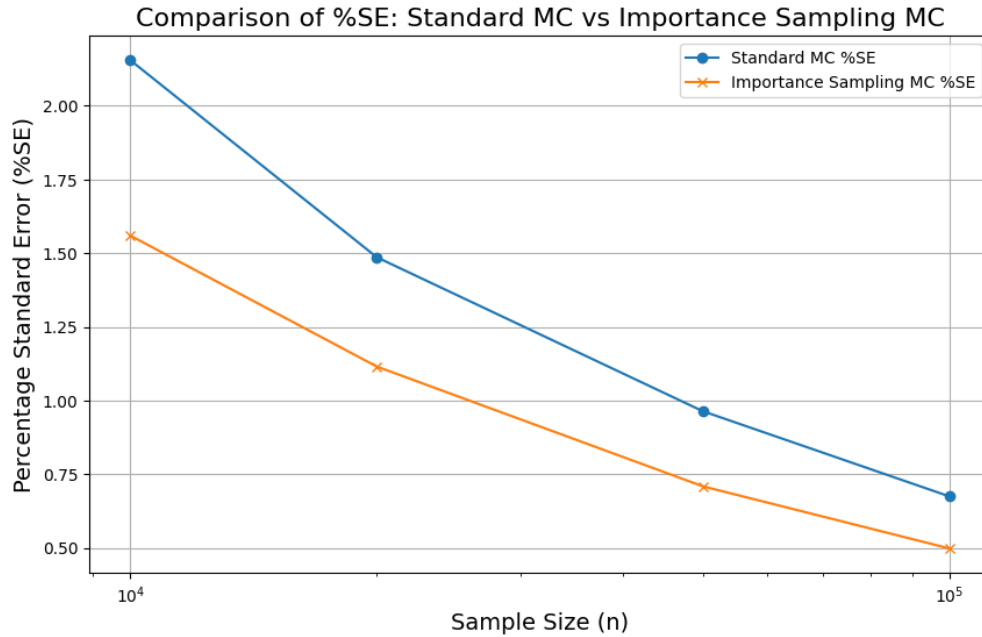


Figure 3: Comparison of Percentage Standard Error (%SE) for Standard Monte Carlo vs Importance Sampling Monte Carlo for Asian Call Option Pricing

Summary

- Standard Monte Carlo is inefficient for rare event option pricing.
- Importance sampling shifts the sampling to important regions.

- For European options, μ centers sampling at the strike boundary.
- For Asian options, a constant shift applies across steps.
- Importance sampling reduces variance and improves efficiency.